```
private readonly CityBreaksContext _context;
private readonly IMemoryCache _cache;
private readonly ILogger<CityService> _logger;

public CityService(CityBreaksContext context, IMemoryCache cache,
➥ ILogger<CityService> logger)
{
    _context = context;
    _cache = cache;
    _logger = logger;
}
```

Add the following new method to the `ICityService` interface, which returns a list of type string:

```
Task<List<string>> GetCityNamesAsync();
```

After adding a stub to the existing `SimpleCityService` (if you haven't removed it already), add an implementation to the `CityService`, which checks the cache and populates it from the database if it is empty (listing 14.25). So you can see what's going on, log different messages based on whether or not the data comes from the cache.

Use the `TryGetValue<TItem>` extension method to retrieve an item with the key specified by the first parameter. Also use the `nameof` operator and pass in the name of the enclosing method instead of using a simple string. This removes the likelihood of getting key names wrong as a result of a typographical error. If the method returns `true`, return the `out` parameter value. Otherwise, retrieve the data from the database and populate the cache using the `Set` method. Convert the city names to lowercase, and perform a comparison in lowercase to prevent casing issues.

Listing 14.25 The `GetCityNamesAsync` method using caching

```
public async Task<List<string>> GetCityNamesAsync()
{
```